MicroGridsPy Planning User Guide

MicroGridsPy Planning

April 13, 2026

Contents

I Shared Platform Workflow **3**

1 Introduction **4**

2 General Workflow **5**

3 Streamlit Interface **6**
3.1 Home . . . 6
3.2 Project Setup . . . 6
3.2.1 Create tab . . . 6
3.2.2 Load tab . . . 6
3.3 Data Audit and Visualization . . . 7
3.4 Model Optimization . . . 7
3.5 Results . . . 7

II Multi-Year Planning Mode **8**

4 Multi-Year Overview **9**

5 Multi-Year Initialization Logic **10**
5.1 Formulation switch . . . 10
5.2 Year labels . . . 10
5.3 Capacity expansion and investment steps . . . 10
5.4 Canonical step-key convention . . . 11
5.5 Scenarios and scenario weights . . . 11

6 Multi-Year Project Directory **12**
6.1 Auto-generated and user-maintained files . . . 12

7 Multi-Year Input Files **13**
7.1 Master file table . . . 13
7.2 `inputs/formulation.json` . . . 14
7.3 `inputs/renewables.yaml` . . . 15
7.4 `inputs/battery.yaml` . . . 17
7.5 `inputs/generator.yaml` . . . 19
7.6 `inputs/grid.yaml` . . . 20
7.7 Informational files . . . 21

8 Multi-Year Time-Series Structure **22**
8.1 `load_demand.csv` . . . 22
8.2 `resource_availability.csv` . . . 22

8.3 grid_import_price.csv and grid_export_price.csv . . . 23
8.4 generator_efficiency_curve.csv . . . 23
8.5 battery_efficiency_curve.csv . . . 23
8.6 battery_calendar_fade_curve.csv . . . 23
8.7 grid_availability.csv . . . 24

9 Multi-Year Conditional Features **25**
9.1 On-grid mode . . . 25
9.2 Export mode . . . 25
9.3 Capacity expansion . . . 25
9.4 Multi-scenario mode . . . 25
9.5 Battery convex loss model . . . 25
9.6 Battery endogenous degradation . . . 25
9.7 Generator efficiency curve . . . 26

10 Multi-Year Consistency Checks **27**

11 Multi-Year Minimal Examples **28**
11.1 Example A: simplest off-grid Multi-Year case . . . 28
11.2 Example B: richer on-grid Multi-Year case . . . 28

III Typical-Year Planning Mode **30**

12 Typical-Year Overview **31**

13 Differences from Multi-Year **32**
13.1 Initialization differences . . . 32
13.2 Investment-step simplification . . . 32
13.3 CSV simplification . . . 32
13.4 YAML simplification . . . 32
13.5 Conditional-feature simplification . . . 33
13.6 Consistency-check differences . . . 33

IV Appendices **34**

A Glossary **35**

B UI Mapping **36**

C Source Basis **37**

Part I

Shared Platform Workflow

Chapter 1

Introduction

MicroGridsPy Planning is the planning module of the MicroGridsPy ecosystem. It is implemented as a Streamlit application backed by Linopy optimization and a project-folder workflow based on JSON, YAML, and CSV files.

The application supports two complementary planning modes:

- **Multi-Year**: an explicit dynamic formulation with year-by-year evolution, investment-step logic, and cohort-style capacity tracking over the planning horizon.
- **Typical-Year**: a reduced steady-state formulation in which one representative operating year is repeated and investment decisions are evaluated in annuity form.

This guide adopts the Multi-Year workflow as the *reference* implementation. Shared workflow, interface, directory, input, and conditional-feature logic are therefore defined once in the Multi-Year chapters. The Typical-Year chapters later describe only the simplifications and differences required when the reduced formulation is selected.

Reference framing

In the current implementation, Multi-Year is the general planning mode and Typical-Year is its reduced special case. When a concept exists in both modes, this guide defines it in the Multi-Year chapters first and then refers back to it from the Typical-Year chapters.

Chapter 2

General Workflow

The minimum workflow for any project is:

1. Launch the Streamlit application and open **Project Setup**.
2. Create a project folder under `projects/<project\_name>/`.
3. Select the planning formulation.
4. Configure the high-level project settings that determine which templates are generated.
5. Click **Initialize project and generate templates**.
6. Edit the generated input files under `projects/<project\_name>/inputs/`.
7. Open **Data Audit and Visualization** to confirm that the dataset loads and the dimensions match the intended study.
8. Open **Model Optimization** and solve the model.
9. Open **Results** to inspect sizing, operation, costs, emissions, and exported files.

This workflow is shared by both planning modes. The main differences lie in how the formulation is initialized and in how years, investment steps, and time-series inputs are organized.

Chapter 3

Streamlit Interface

3.1 Home

`Home.py` is the landing page. It links to:

- `pages/0\_Project\_Setup.py`
- `pages/1\_Data\_Audit\_and\_Visualization.py`
- `pages/3\_Optimization.py`
- `pages/4\_Results.py`

The Home page introduces the two formulations at a high level but does not configure the model directly.

3.2 Project Setup

This page creates or loads projects.

3.2.1 Create tab

The Create tab controls:

- project name and description,
- formulation selection,
- grid and export settings,
- renewable-resource count and labels,
- uncertainty settings,
- battery and generator advanced-model options,
- optimization constraints,
- template generation.

3.2.2 Load tab

The Load tab marks an existing project as active without regenerating inputs.

Important behavior

If a project is created with a name that already exists, the generated files in `inputs/` are overwritten using the current Project Setup selections. Existing user-edited inputs should therefore be backed up or reviewed before regeneration.

3.3 Data Audit and Visualization

This page reads `inputs/formulation.json`, initializes the formulation-specific sets, loads the canonical project dataset, and displays:

- required and optional input-file status,
- dataset-loading status,
- coordinate summaries,
- parameter summaries,
- optimization-constraint summaries,
- advanced-curve diagnostics,
- grid-availability controls for on-grid projects,
- time-series plots.

For on-grid projects, `grid\_availability.csv` is treated as a derived artifact and can be regenerated from `grid.yaml`.

3.4 Model Optimization

This page:

- requires an active project,
- reads `inputs/formulation.json`,
- builds the formulation-specific optimization model,
- lets the user select `highs` or `gurobi`,
- optionally writes a problem file,
- stores a solver log under `projects/<project\_name>/logs/`,
- solves the single-objective planning problem.

3.5 Results

This page dispatches to the formulation-specific results renderer. Depending on the active mode, it displays sizing, key performance indicators, operational summaries, economic breakdowns, and exported CSV or Excel artifacts.

Part II

Multi-Year Planning Mode

Chapter 4

Multi-Year Overview

Multi-Year is implemented as `core_formulation = "dynamic"`. It is the general planning workflow in the current codebase.

Compared with Typical-Year, Multi-Year is structurally richer because it makes the following dimensions explicit:

- `year`: modeled years over the project horizon,
- `inv_step`: investment steps and commissioning cohorts,
- `scenario`: uncertainty branches,
- `resource`: renewable-resource labels,
- `period`: hourly periods within the representative operating year.

The Multi-Year formulation should be used when the study requires explicit year evolution, time-dependent fuel prices or grid conditions, investment sequencing, battery degradation tracking, or cohort-aware capacity accounting.

Chapter 5

Multi-Year Initialization Logic

5.1 Formulation switch

Project Setup activates Multi-Year by writing `core_formulation = "dynamic"` to `inputs/formulation.json`.

The Multi-Year set initializer then derives:

- `period = 0,...,8759`,
- `year = [start_year_label, start_year_label + 1, ...]`,
- `scenario = scenario labels`,
- `resource = renewable-resource labels`,
- `inv_step = [1]` if capacity expansion is disabled,
- `inv_step = [1,2,...,N]` if capacity expansion is enabled.

5.2 Year labels

The current implementation requires:

- `start_year_label` to be integer-like,
- `time_horizon_years` to be a positive integer.

If `start_year_label = 2026` and `time_horizon_years = 10`, the modeled years are: `2026, 2027, ..., 2035`.

Those labels are then reused consistently in CSV headers, yearly fuel-price lists, grid-connection timing, and yearly result exports.

5.3 Capacity expansion and investment steps

If `capacity_expansion = false`, there is one investment step only and generated YAML templates use a single `"1"` block under `investment.by_step`.

If `capacity_expansion = true`, the UI requires `investment_steps_years`, for example `[5,5]`. The initializer then constructs:

- `inv_step = [1,2]`,
- `inv_step_start_year = [2026,2031]`,
- `inv_step_end_year = [2030,2035]`,

- `inv_step_len_years = [5,5]`,
- `year_inv_step[year]`,
- `inv_active_in_year[inv_step, year]`.

Capacity expansion and cohorts

In the current backend, investment-side parameters may vary by `inv_step`, but technical parameters remain shared across steps. Capacity commissioned in a step becomes a cohort or vintage tracked through later years, while the underlying technology definition stays shared.

5.4 Canonical step-key convention

The current user-facing Multi-Year templates use canonical numeric step labels:

- `"1"` for the single-step case,
- `"1"`, `"2"`, ... for multi-step cases.

The loader still accepts lightweight legacy aliases such as `step_1` and, for single-step projects only, `base`. New inputs should follow the numeric convention directly.

5.5 Scenarios and scenario weights

If multi-scenario mode is disabled, the application uses a single scenario, typically `scenario_1`. If it is enabled, `multi_scenario.scenario_labels` defines the ordered labels and `multi_scenario.scenario_weig` defines the scenario weights. All scenario-dependent CSV headers and YAML mappings must use the same labels.

Chapter 6

Multi-Year Project Directory

The representative Multi-Year project structure is:

```
projects/
  <project_name>/
    inputs/
      formulation.json
      README_inputs.md
      load_demand.csv
      resource_availability.csv
      renewables.yaml
      battery.yaml
      generator.yaml
      grid.yaml                       # only if on-grid
      grid_import_price.csv           # only if on-grid
      grid_export_price.csv           # only if on-grid and export enabled
      grid_availability.csv           # derived, backend-generated
      battery_efficiency_curve.csv    # only if convex battery loss is enabled
      battery_calendar_fade_curve.csv # only if calendar fade is enabled
      generator_efficiency_curve.csv  # only if generator curve mode is enabled
    logs/
    results/
      multi_year/
        dispatch_timeseries.csv
        energy_balance.csv
        design_by_step.csv
        kpis_yearly.csv
        cashflows_discounted.csv
        scenario_costs_yearly.csv
        results_multi_year.xlsx
```

6.1 Auto-generated and user-maintained files

- Auto-generated: `formulation.json`, `README\_inputs.md`, the primary input templates, solver logs, and result exports.
- Derived: `grid\_availability.csv` for on-grid projects.
- User-edited: the primary input templates under `inputs/`, except `grid\_availability.csv`.

Chapter 7

Multi-Year Input Files

7.1 Master file table

File	Format	Mandatory	When used	Edited by user	Dimensional meaning
`formulation.json`	JSON	Yes	always	normally no	project-level flags, horizon, scenarios, constraints
`load_demand.csv`	CSV	Yes	always	Yes	hourly demand by scenario and year
`resource_availability.csv`	CSV	Yes	always	Yes	hourly renewable availability by scenario, year, and resource
`renewables.yaml`	YAML	Yes	always	Yes	renewable investment by step and shared technical data
`battery.yaml`	YAML	Yes	always	Yes	battery investment by step and shared technical and degradation data
`generator.yaml`	YAML	Yes	always	Yes	generator investment by step, shared technical data, yearly fuel prices
`grid.yaml`	YAML	Conditional	on-grid only	Yes	grid line, outages, renewable share, emissions, connection timing
`grid_import_price.csv`	CSV	Conditional	on-grid only	Yes	hourly import tariff by scenario and year
`grid_export_price.csv`	CSV	Conditional	export enabled	Yes	hourly export tariff by scenario and year
`grid_availability.csv`	CSV	Derived	on-grid only	No	backend-generated hourly availability by scenario and year
`battery_efficiency_curve.csv`	CSV	Conditional	convex battery loss	Yes	battery power-efficiency curve

File	Format	Mandatory	When used	Edited by user	Dimensional meaning
`battery_calendar_fade_curve.csv`	CSV	Conditional	calendar fade enabled	Yes	calendar-ageing curve by average yearly state of charge
`generator_efficiency_curve.csv`	CSV	Conditional	generator curve mode	Yes	generator partial-load efficiency curve
`README_inputs.md`	Markdown	No	always	No	human-readable summary only

7.2 `inputs/formulation.json`

This file is written by Project Setup and read by the pages, set initializer, and Multi-Year data loader.

Path	Meaning	Type	Unit	Notes
`core_formulation`	formulation switch	string	–	must be `"dynamic"`
`system_type`	UI-level system type	string	–	`"off_grid"` or `"on_grid"`
`on_grid`	grid activation flag	bool	–	used directly by loader logic
`grid_allow_export`	export activation flag	bool	–	controls export variables and file generation
`unit_commitment`	integer sizing toggle	bool	–	in current code this means integer sizing, not chronological unit commitment
`start_year_label`	first modeled year label	int-like string or int	year	must be parseable as a positive integer
`time_horizon_years`	modeled horizon length	int	years	positive integer
`social_discount_rate`	social discount rate	number	share	used in objective accounting and reporting
`capacity_expansion`	capacity-expansion toggle	bool	–	activates multiple investment steps
`investment_steps_years`	ordered list of step durations	list[int] or null	years	required when capacity expansion is enabled; sum must equal horizon length
`multi_scenario.enabled`	scenario replication flag	bool	–	false means one scenario

Path	Meaning	Type	Unit	Notes
`multi_scenario.n_scenarios`	number of scenarios	int	–	should match labels and weights
`multi_scenario.scenario_labels`	scenario labels	list[str]	–	must match CSV headers and YAML mappings
`multi_scenario.scenario_weights`	scenario weights	list[number]	share	normalized by the loader
`optimization_constraints.enforcement`	scenario-enforcement mode	string	–	`"scenario_wise"` or `"expected"`
`optimization_constraints.min_renewable_penetration`	renewable-penetration floor	number	share	typically between 0 and 1
`optimization_constraints.max_lost_load_fraction`	lost-load fraction	number	share	typically between 0 and 1
`optimization_constraints.lost_load_cost_per_kwh`	lost-load penalty	number	currency/kWh	zero disables the economic penalty
`optimization_constraints.land_availability_m2`	land capacity	number or null	m^2	null disables the land constraint
`optimization_constraints.emissions_cost_per_kgco2e`	emissions price	number	currency/kg CO_2e	applied in objective and reporting
`system_configuration.n_sources`	renewable-resource count	int	–	should match generated renewable entries
`battery_model.loss_model`	battery loss formulation	string	–	`"constant_efficiency"` or `"convex_loss_epigraph"`
`battery_model.degradation_model.cycle_fade_enabled`	cycle-fade toggle	bool	–	valid in Multi-Year only with convex battery loss
`battery_model.degradation_model.calendar_fade_enabled`	calendar-fade toggle	bool	–	valid in Multi-Year only with convex battery loss
`battery_model.degradation_model` other detailed fields	legacy detailed degradation inputs	mixed	mixed	current UI no longer writes battery-specific degradation data here; current workflow stores them in `battery.yaml`
`generator_model.efficiency_model`	generator part-load formulation	string	–	`"constant_efficiency"` or `"efficiency_curve"`

7.3 `inputs/renewables.yaml`

Generated schema:

```yaml
renewables:
  - id: res_1
    conversion_technology: Solar_PV
    resource: Solar
    investment:
      by_step:
        "1":
          nominal_capacity_kw: 1.0
          specific_investment_cost_per_kw: 0.0
          wacc: 0.0
          grant_share_of_capex: 0.0
          lifetime_years: 25
          embedded_emissions_kgco2e_per_kw: 0.0
          fixed_om_share_per_year: 0.0
          production_subsidy_per_kwh: 0.0
    technical:
      inverter_efficiency: 1.0
      specific_area_m2_per_kw: null
      max_installable_capacity_kw: null
      capacity_degradation_rate_per_year: 0.0
```

One entry is required for each renewable resource. The field `resource` must match the third header level of `resource\_availability.csv`.

Path	Meaning	Type	Unit	Behavior and notes
`renewables[].id`	internal identifier	string	–	informational only
`renewables[].conversion_technology`	technology label	string	–	shared across steps and scenarios
`renewables[].resource`	resource label	string	–	must match `resource\_availability.csv` and the `resource` coordinate
`renewables[].investment.by_step.<step>.nominal_capacity_kw`	capacity installed in step <step>	number	kW	step-dependent
`renewables[].investment.by_step.<step>.specific_investment_cost_per_kw`	CAPEX intensity	number	currency/kW	step-dependent; annualized with WACC and lifetime
`renewables[].investment.by_step.<step>.wacc`	WACC	number	share	step-dependent
`renewables[].investment.by_step.<step>.grant_share_of_capex`	grant share of capex	number	share	step-dependent
`renewables[].investment.by_step.<step>.lifetime_years`	lifetime	number	years	step-dependent; used by replacement logic
`renewables[].investment.by_step.<step>.embedded_emissions_kgco2e_per_kw`	embedded emissions	number	$kgCO_2e/kW$	step-dependent
`renewables[].investment.by_step.<step>.fixed_om_share_per_year`	fixed O&M share per year	number	share/year	optional; defaults to 0.0 if omitted
`renewables[].investment.by_step.<step>.production_subsidy_per_kwh`	production subsidy	number	currency/kWh	optional; defaults to 0.0 if omitted
`renewables[].technical.inverter_efficiency`	inverter efficiency	number	–	shared; usually in $(0, 1]$

Path	Meaning	Type	Unit	Behavior and notes
`renewables[].technical.specific_area_m2_per_kw`	land area per installed unit	number or null	m^2/kW	shared; needed in practice when land constraint is active
`renewables[].technical.max_installable_capacity_kw`	shared capacity cap	number or null	kW	shared; null means no explicit cap
`renewables[].technical.capacity_degradation_rate_per_year`	yearly capacity fade	number	1/year	optional; defaults to 0.0

The current Multi-Year parser accepts the shared-technology schema only. Legacy branches such as top-level `by_step`, `technical.by_step`, and `operation` are no longer part of the current user workflow.

7.4 inputs/battery.yaml

Generated schema:

```
battery:
  label: Battery
  investment:
    by_step:
      "1":
        nominal_capacity_kwh: 1.0
        specific_investment_cost_per_kwh: 0.0
        wacc: 0.0
        calendar_lifetime_years: 10
        embedded_emissions_kgco2e_per_kwh: 0.0
        fixed_om_share_per_year: 0.0
  technical:
    charge_efficiency: 0.95
    discharge_efficiency: 0.96
    initial_soc: 0.5
    depth_of_discharge: 0.8
    max_discharge_time_hours: 5.0
    max_charge_time_hours: 5.0
    max_installable_capacity_kwh: null
    efficiency_curve_csv: battery_efficiency_curve.csv
    capacity_degradation_rate_per_year: 0.0
    initial_soh: 1.0
    end_of_life_soh: 0.8
    cycle_lifetime_to_eol_cycles: 6000.0
    calendar_fade_curve_csv: battery_calendar_fade_curve.csv
    calendar_time_increment_per_year: 1.0
```

Path	Meaning	Type	Unit	Behavior and notes
`battery.label`	battery label	string	–	reporting only
`battery.investment.by_step.<step>.nominal_capacity_kwh`	nominal capacity per installed unit	number	kWh	step-dependent
`battery.investment.by_step.<step>.specific_investment_cost_per_kwh`	CAPEX intensity	number	currency/kWh	step-dependent; annualized with WACC and lifetime
`battery.investment.by_step.<step>.wacc`	WACC	number	share	step-dependent

Path	Meaning	Type	Unit	Behavior and notes
`battery.investment.by_step.<step>.calendar_lifetime_years`	battery calendar lifetime in years	number	years	step-dependent
`battery.investment.by_step.<step>.embodied_emissions_kgco2e_per_kwh`	embodied emissions	number	kgCO2e/kWh	step-dependent
`battery.investment.by_step.<step>.fixed_om_share_per_year`	fixed O&M share per year	number	share/year	optional; defaults to 0.0 if omitted
`battery.technical.charge_efficiency`	one-way charge efficiency	number	–	shared across steps
`battery.technical.discharge_efficiency`	one-way discharge efficiency	number	–	shared across steps
`battery.technical.initial_soc`	initial state of charge	number	share	shared
`battery.technical.initial_soh`	initial state of health	number	share	required when endogenous degradation is enabled in current generated workflow
`battery.technical.depth_of_discharge`	usable fraction of nominal capacity	number	share	shared
`battery.technical.max_discharge_time_hours`	discharge time proxy	number	h	shared
`battery.technical.max_charge_time_hours`	charge time proxy	number	h	shared
`battery.technical.max_installable_capacity_kwh`	installable capacity	number or null	kWh	shared; null means no explicit cap
`battery.technical.efficiency_curve_csv`	battery loss-curve filename	string or null	path	required when convex battery loss is selected
`battery.technical.capacity_degradation_rate_per_year`	capacity fade	number	1/year	used unless calendar fade is enabled; then internally suppressed
`battery.technical.end_of_life_soh`	end-of-life state of health	number	share	conditional; used for endogenous degradation
`battery.technical.cycle_lifetime_to_eol_cycles`	cycle life to end of life	number	cycles	conditional; used when cycle fade is enabled
`battery.technical.calendar_fade_curve_csv`	calendar fade curve filename	string	path	conditional; used when calendar fade is enabled
`battery.technical.calendar_time_increment_per_year`	calendar time increment	number	increment/year	conditional; current code supports constant-per-year mode only

Battery degradation ownership

The current user-facing workflow separates high-level activation flags from battery-specific degradation data. `formulation.json` activates cycle fade and calendar fade, while `battery.yaml` stores the battery's actual degradation parameters such as `initial_soh`, end-of-life state of health, cycle lifetime, and calendar-fade file paths.

7.5 inputs/generator.yaml

Generated schema:

```
generator:
  label: Generator
  investment:
    by_step:
      "1":
        nominal_capacity_kw: 1.0
        lifetime_years: 10
        specific_investment_cost_per_kw: 0.0
        wacc: 0.0
        embedded_emissions_kgco2e_per_kw: 0.0
        fixed_om_share_per_year: 0.0
  technical:
    nominal_efficiency_full_load: 0.30
    efficiency_curve_csv: generator_efficiency_curve.csv
    max_installable_capacity_kw: null
    capacity_degradation_rate_per_year: 0.0
fuel:
  label: Fuel
  technical:
    lhv_kwh_per_unit_fuel: 0.0
    direct_emissions_kgco2e_per_unit_fuel: 0.0
  cost:
    by_scenario:
      scenario_1:
        by_year_cost_per_unit_fuel: [0.0, 0.0]
```

Path	Meaning	Type	Unit	Behavior and notes
generator.label	generator label	string	–	reporting only
generator.investment.by_step.<step>.nominal_capacity_kw	step-dependent installed unit capacity	number	kW	step-dependent
generator.investment.by_step.<step>.lifetime_years	step-dependent lifetime	number	years	step-dependent
generator.investment.by_step.<step>.specific_investment_cost_per_kw	step-dependent CAPEX	number	currency/kW	step-dependent
generator.investment.by_step.<step>.wacc	step-dependent WACC	number	share	step-dependent
generator.investment.by_step.<step>.embedded_emissions_kgco2e_per_kw	step-dependent embodied emissions	number	$kgCO_2e$/kW	step-dependent
generator.investment.by_step.<step>.fixed_om_share_per_year	step-dependent fixed O&M share	number	share/year	optional; defaults to 0.0 if omitted
generator.technical.nominal_efficiency_full_load	full-load efficiency	number	–	shared across steps
generator.technical.efficiency_curve_csv	curve filename	string or null	path	conditional; used when generator curve mode is active
generator.technical.max_installable_capacity_kw	installable capacity	number or null	kW	shared; null means no explicit cap
generator.technical.capacity_degradation_rate_per_year	degradation rate per year	number	1/year	optional; defaults to 0.0
fuel.label	fuel label	string	–	reporting only
fuel.technical.lhv_kwh_per_unit_fuel	lower heating value	number	kWh/unit fuel	shared across steps and scenarios
fuel.technical.direct_emissions_kgco2e_per_unit_fuel	direct emission factor	number	$kgCO_2e$/unit fuel	shared across steps and scenarios

Path	Meaning	Type	Unit	Behavior and notes
`fuel.cost.by_scenario.<scenario>.by_year.fuel_cost_per_unit_fuel`	yearly fuel-cost trajectory	list[number]	currency/unit fuel	scenario- and year-dependent; list length must equal horizon length

7.6 inputs/grid.yaml

This file exists only if `on_grid = true`.

```
grid:
  by_scenario:
    scenario_1:
      line:
        capacity_kw: 0.0
        transmission_efficiency: 1.0
        renewable_share: 0.0
        emissions_factor_kgco2e_per_kwh: 0.0
      outages:
        average_outages_per_year: 0.0
        average_outage_duration_minutes: 0.0
        outage_scale_od_hours: 0.6
        outage_shape_od: 0.56
        outage_seed: 0
      first_year_connection: 2026
```

Path	Meaning	Type	Unit	Behavior and notes
`grid.by_scenario.<scenario>.line.capacity_kw`	interconnection capacity	number	kW	scenario-dependent; same limit used for import and export
`grid.by_scenario.<scenario>.line.transmission_efficiency`	delivery efficiency	number	–	scenario-dependent; must stay in $[0, 1]$
`grid.by_scenario.<scenario>.line.renewable_share`	renewable share of imported power	number	share	scenario-dependent; must stay in $[0, 1]$
`grid.by_scenario.<scenario>.line.emissions_factor_kgco2e_per_kwh`	grid emissions factor	number	kgCO2e/kWh	scenario-dependent; non-negative
`grid.by_scenario.<scenario>.outages.average_outages_per_year`	average outages per year	number	events/year	used by outage simulator
`grid.by_scenario.<scenario>.outages.average_outage_duration_minutes`	average outage duration	number	minutes	used by outage simulator
`grid.by_scenario.<scenario>.outages.outage_scale_od_hours`	Weibull outage scale parameter	number	h	must be strictly positive
`grid.by_scenario.<scenario>.outages.outage_shape_od`	Weibull outage shape parameter	number	–	must be strictly positive
`grid.by_scenario.<scenario>.outages.outage_seed`	outage seed	int-like number	seed	used for deterministic regeneration
`grid.by_scenario.<scenario>.first_year_connection`	first connected modeled year	year label, int, null, or blank	year	null or blank means connected from horizon start

Connection timing

The preferred input for `first_year_connection` is an exact label from the modeled `year` coordinate, for example `2031`. If it is after the last modeled year, grid availability remains zero for the whole horizon.

7.7 Informational files

`README\_inputs.md` is generated for human guidance only and is not parsed as a primary model input. YAML `meta` blocks, where present, are descriptive rather than structural.

Chapter 8

Multi-Year Time-Series Structure

8.1 `load_demand.csv`

This file uses a two-row header:

- row 1: scenario,
- row 2: year.

The first column must be `meta/hour`. All scenario-year combinations must exist. The file must contain exactly 8760 rows and `meta/hour` must run from 0 to 8759. Values are hourly demand in kWh.

```
meta,scenario_1,scenario_1
hour,2026,2027
0,15.2,16.0
1,14.9,15.8
```

The loaded array has dimensions `(year, period, scenario)`.

8.2 `resource_availability.csv`

This file uses a three-row header:

- row 1: scenario,
- row 2: year,
- row 3: resource.

The first column is `("meta","hour","")`. All scenario-year-resource combinations must exist. The file must contain exactly 8760 rows. Values are intended as capacity factors close to the interval $[0, 1]$, although the current loader tolerates a broader numeric bound.

```
meta,scenario_1,scenario_1
hour,2026,2026
,Solar,Wind
0,0.00,0.45
1,0.00,0.43
```

The loaded array has dimensions `(year, period, scenario, resource)`.

8.3 grid_import_price.csv and grid_export_price.csv

These files follow the same two-row scenario/year header used by load_demand.csv. They are required only for on-grid projects, with grid_export_price.csv additionally conditional on export being enabled. Values are expressed in currency per kWh.

8.4 generator_efficiency_curve.csv

Required columns are:

- Relative Power Output [-]
- Efficiency [-]

Current accepted behavior:

- an explicit 0.0 row is optional,
- at least one positive-load point is required,
- the last relative-power point must be 1.0,
- positive-load points must be strictly increasing,
- efficiency values for positive-load points must be strictly positive.

The preferred interpretation is that Efficiency [-] is a normalized multiplier relative to generator.technical
Legacy absolute-efficiency curves are still accepted where supported by the parser.

8.5 battery_efficiency_curve.csv

Required columns are:

- relative_power_pu
- charge_efficiency
- discharge_efficiency

The file must contain at least two rows, relative_power_pu must be strictly increasing in the interval $(0, 1]$, and the last point must be 1.0. The preferred interpretation is that the charge and discharge columns are normalized multipliers around the scalar efficiencies declared in battery.yaml.

8.6 battery_calendar_fade_curve.csv

Required columns are:

- soc_pu
- preferred: calendar_fade_coefficient_per_year
- legacy alias: calendar_fade_coefficient_per_step

The file must contain at least two rows. soc_pu must lie in $[0, 1]$. If the first point is above zero, the loader prepends a zero point automatically. The last soc_pu point must be 1.0. The coefficient is interpreted as a yearly calendar-ageing coefficient used together with battery_calendar_time_incremen

8.7 `grid_availability.csv`

This file is derived rather than user-authored. It is regenerated from `grid.yaml` and the current Multi-Year sets. It uses the same two-row scenario/year header as the price files and represents binary hourly availability.

Chapter 9

Multi-Year Conditional Features

9.1 On-grid mode

Activated by `on_grid = true`. Additional required inputs are:

- `grid.yaml`
- `grid\_import\_price.csv`
- `grid\_export\_price.csv` if export is enabled

The derived file is `grid\_availability.csv`.

9.2 Export mode

Activated by `grid_allow_export = true`. This adds `grid\_export\_price.csv` and activates export-side variables in the optimization model.

9.3 Capacity expansion

Activated by `capacity_expansion = true`. Renewable, battery, and generator templates then generate one `investment.by_step` block per step, and the model tracks each commissioned cohort by `inv_step`.

9.4 Multi-scenario mode

Activated by `multi_scenario.enabled = true`. All hourly time-series CSVs must then contain every scenario-year combination. `fuel.cost.by_scenario` and `grid.by_scenario` must also cover every scenario label.

9.5 Battery convex loss model

Activated by `battery_model.loss_model = "convex_loss_epigraph"`. This requires `battery\_efficiency\_curve.csv` and switches the battery-loss representation from scalar efficiencies to a convex piecewise-linear surrogate based on DC-side relative power.

9.6 Battery endogenous degradation

Activated by convex battery loss together with either `cycle_fade_enabled` or `calendar_fade_enabled`. Additional battery inputs may include:

- `initial_soh`,
- `end_of_life_soh`,
- `cycle_lifetime_to_eol_cycles`,
- `battery\_calendar\_fade\_curve.csv`,
- `calendar_time_increment_per_year`.

Cycle fade and calendar fade are tracked as yearly degradation state variables. Replacement timing still follows the declared battery calendar lifetime.

9.7 Generator efficiency curve

Activated by `generator_model.efficiency_model = "efficiency_curve"`. This requires `generator\_efficiency\_curve.csv` and converts the user curve into an internal convex fuel-use surrogate.

Chapter 10

Multi-Year Consistency Checks

Before solving, verify the following:

1. `start_year_label` is an integer-like calendar year.
2. If capacity expansion is enabled, `investment_steps_years` sums exactly to `time_horizon_years`.
3. Scenario labels match in `formulation.json`, CSV headers, `fuel.cost.by_scenario`, and `grid.by_scenario`.
4. Resource labels match between `renewables[].resource` and the third header row of `resource\_availability.csv`.
5. `fuel.cost.by_scenario.<scenario>.by_year_cost_per_unit_fuel` contains one value for every modeled year.
6. All hourly files contain exactly 8760 rows and the correct `meta/hour` sequence from 0 to 8759.
7. If on-grid mode is active, `first_year_connection` uses an exact modeled year label whenever possible.
8. If land constraints are active, every renewable entry provides a meaningful `specific_area_m2_per_kw`.
9. If convex battery loss is active, `battery.technical.efficiency_curve_csv` points to an existing curve file.
10. If battery calendar fade is enabled, `battery\_calendar\_fade\_curve.csv` is present and the calendar time increment uses the supported constant-per-year semantics.
11. If generator curve mode is active, the generator curve ends at relative power `1.0`.
12. Do not try to make technical parameters vary by investment step. The current Multi-Year backend assumes shared technical parameters across steps.

Chapter 11

Multi-Year Minimal Examples

11.1 Example A: simplest off-grid Multi-Year case

This is the smallest dynamic project that still exercises the Multi-Year machinery:

- `core_formulation = "dynamic"`
- `system_type = "off_grid"`
- `on_grid = false`
- `capacity_expansion = false`
- `start_year_label = 2026`
- `time_horizon_years = 10`
- `multi_scenario.enabled = false`
- one renewable resource
- battery loss model = `constant_efficiency`
- generator efficiency model = `constant_efficiency`

Required files are `formulation.json`, `load\_demand.csv`, `resource\_availability.csv`, `renewables.yaml`, `battery.yaml`, and `generator.yaml`. In the single-step case, each component uses one `"1"` investment block.

11.2 Example B: richer on-grid Multi-Year case

This configuration exercises most dynamic features:

- `core_formulation = "dynamic"`
- `system_type = "on_grid"`
- `on_grid = true`
- `grid_allow_export = true`
- `capacity_expansion = true`
- `investment_steps_years = [5,5]`
- `start_year_label = 2026`
- `time_horizon_years = 10`
- optional multiple scenarios

- optional convex battery loss and endogenous degradation
- optional generator efficiency curve

Additional required files are `grid.yaml`, `grid\_import\_price.csv`, `grid\_export\_price.csv`, and any active advanced-curve files.

Part III

Typical-Year Planning Mode

Chapter 12

Typical-Year Overview

Typical-Year is implemented as `core_formulation = "steady_state"`. It uses the same application pages and the same general project workflow defined in Part I, but it collapses the dynamic structure into a representative-year planning problem.

Conceptually, Typical-Year is a reduced special case of the Multi-Year workflow described in Part II:

- the operating year is represented once and repeated conceptually,
- investment costs are annualized directly,
- the objective minimizes expected equivalent annual cost,
- scenarios may differ operationally while sharing the same design decisions.

Typical-Year simplification

Typical-Year follows the same platform workflow, file layout, and component logic described for Multi-Year, but removes explicit year evolution, explicit investment-step dynamics, and battery-degradation state tracking across years.

Chapter 13

Differences from Multi-Year

This chapter lists only the differences from the reference Multi-Year workflow in Sections 5 to 11.

13.1 Initialization differences

When Typical-Year is selected, Project Setup writes:

- `core_formulation = "steady_state"`,
- `start_year_label = "typical_year"`.

There is no dynamic horizon and no explicit Multi-Year cohort accounting in the optimization path. The representative year is indexed by the single label `typical_year`.

13.2 Investment-step simplification

Typical-Year still uses the same YAML nesting concept, namely `investment.by_step`, but only one step is relevant. Current Typical-Year templates keep the single-step key `base`. The Typical-Year parsers also accept a single non-`base` step block when only one block is present.

13.3 CSV simplification

Typical-Year uses the same CSV file families described in Chapter 8, with the following simplifications:

- the year header is the single label `typical_year`,
- there is no explicit dynamic year expansion,
- fuel prices are scalar scenario values rather than yearly trajectories,
- grid connection timing by explicit year does not apply.

13.4 YAML simplification

Typical-Year reuses the same component files described in Chapter 7, with the following simplifications:

- investment data are single-step rather than step-dependent over a horizon,
- battery degradation inputs such as `initial_soh`, calendar-fade curves, and endogenous year-to-year degradation states are not part of the current Typical-Year workflow,
- generator fuel data use scenario-based scalar fuel costs rather than yearly price lists,

- `first_year_connection` is not used because the representative-year formulation does not model explicit calendar years.

13.5 Conditional-feature simplification

Most conditional features remain conceptually the same as in Chapter 9. The main formulation-specific simplifications are:

- convex battery loss remains available,
- generator efficiency curves remain available,
- explicit battery endogenous degradation is not part of the current Typical-Year user workflow,
- capacity expansion and year-dependent cohort logic are not active in the optimization path.

13.6 Consistency-check differences

In addition to the shared file-consistency checks inherited from Chapter 10, the Typical-Year user should verify:

- all time-series files use the single year header `typical_year`,
- only one investment block is provided in each component file,
- on-grid projects provide the required grid-price templates but do not attempt to declare dynamic first-connection years,
- dynamic battery-degradation inputs are not introduced into Typical-Year files.

Part IV

Appendices

Appendix A

Glossary

Term	Meaning in current implementation
`year`	explicit modeled year label, usually a calendar year such as `2026`; in Typical-Year this collapses to `typical_year`
`period`	hourly index within the representative operating year, `0,...,8759`
`scenario`	uncertainty branch with shared design decisions and different operating conditions
`inv_step`	investment step and commissioning cohort index
`resource`	renewable-resource label used in sets and time-series headers
cohort / vintage	capacity commissioned in an investment step and then tracked through later years
shared-technology model	current Multi-Year interpretation in which investment terms may vary by step but technical parameters remain shared
capacity expansion	activation of multiple investment steps over a dynamic horizon

Appendix B

UI Mapping

UI choice	Effect
Multi-Year formulation	writes `core_formulation = "dynamic"`
Typical-Year formulation	writes `core_formulation = "steady_state"`
Start year and horizon	set the Multi-Year `year` coordinate and year headers
Capacity expansion enabled	generates multiple `investment.by_step` blocks in Multi-Year
On-grid mode	generates `grid.yaml` and `grid\_import\_price.csv`
Export enabled	also generates `grid\_export\_price.csv`
Multi-scenario mode	replicates scenario labels across CSV headers and scenario-keyed YAML mappings
Battery loss model = `convex_loss_epigraph`	generates the battery efficiency-curve template and activates curve-based loader logic
Battery calendar fade enabled	generates the calendar-fade curve template and writes calendar-fade battery fields
Generator efficiency model = `efficiency_curve`	generates the generator efficiency-curve template and writes its filename into `generator.yaml`

Appendix C

Source Basis

This unified guide was reconstructed from the current repository implementation, especially:

- `pages/0\_Project\_Setup.py`
- `pages/1\_Data\_Audit\_and\_Visualization.py`
- `pages/3\_Optimization.py`
- `pages/4\_Results.py`
- `core/io/templates.py`
- `core/multi\_year\_model/sets.py`
- `core/multi\_year\_model/data.py`
- `core/data\_pipeline/battery\_loss\_model.py`
- `core/data\_pipeline/battery\_degradation\_model.py`
- `core/data\_pipeline/battery\_calendar\_fade\_model.py`
- `core/data\_pipeline/generator\_partial\_load\_model.py`

When code, template comments, and older prose diverged, this guide followed the current implementation and the current generated-template workflow.